

26/8/26 DAY 3:

Functions

Functions Declaration/Prototype, Call, Definition

This all we have discussed in day 2.

```
#include <iostream>
using namespace std;

//returnType functionName(parameters...){
//
//}

// function without parameters and without returntype
void sayHello(){
    cout << "Hello" << endl;
}

// function without parameters and with returntype
int returnValue(){
    return 100;
}

void add(int,int); // function declaration/prototype
int multiply(int a, int b);

int main() {

    sayHello(); // function call

    int val = returnValue();

    cout << val << endl;

    add(10,20); // arguments

    int result = multiply(30, 40);

    cout << result << endl;

    return 0;
}

// function with parameters and without returntype
void add(int a, int b){
    cout << "Sum : " << a + b << endl;
}
```

```
// function with parameters and with returntype
int multiply(int a, int b){
    return a * b;
}
```

Call by Value

- In call by value, a copy of the actual parameter is passed to the function.
- The changes made to the parameter inside the function do not affect the actual parameter.

```
#include <iostream>
using namespace std;
void swap(int a, int b){
    int temp = a;
    a = b;
    b = temp;
}

int main() {
    int x = 10, y = 20;
    cout << "Before swap: x = " << x << ", y = " << y << endl;
    swap(x, y); // passing arguments by value
    cout << "After swap: x = " << x << ", y = " << y << endl;    // values of x and
y remain unchanged
    return 0;
}
```

Output:

```
Before swap: x = 10, y = 20
After swap: x = 10, y = 20
```

Call by Reference

- In call by reference, the address of the actual parameter is passed to the function.
- The changes made to the parameter inside the function affect the actual parameter.

```
#include <iostream>
using namespace std;

void swap(int &a, int &b){ // reference parameters
    int temp = a;
    a = b;
    b = temp;
}

int main() {
```

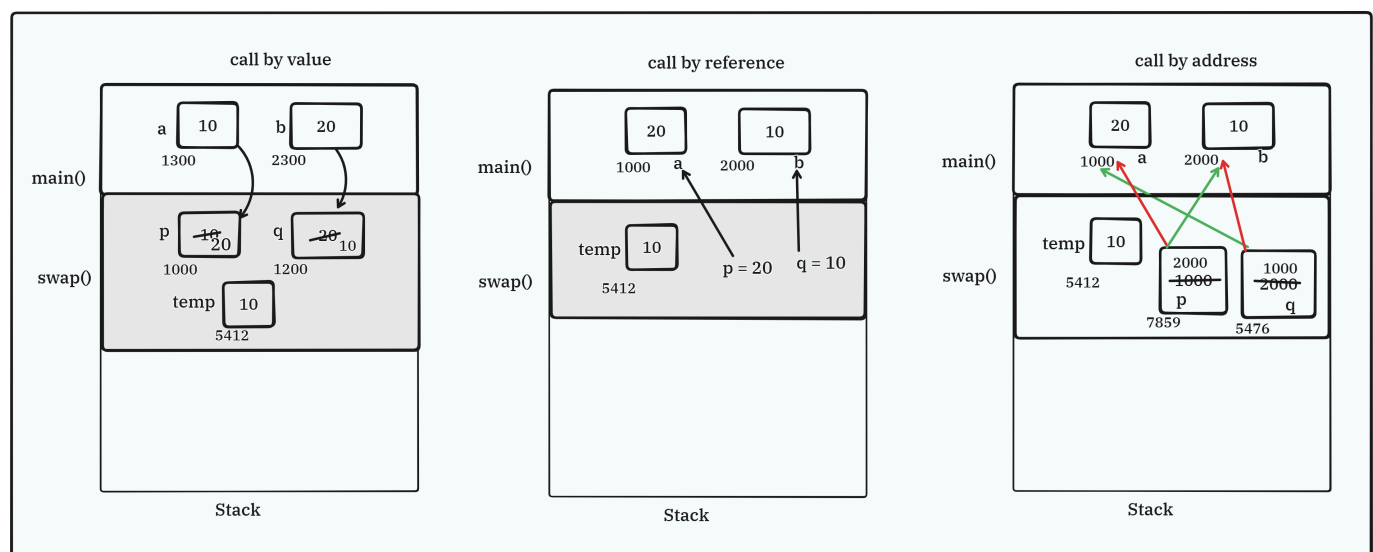
```

int x = 10, y = 20;
cout << "Before swap: x = " << x << ", y = " << y << endl;
swap(x, y); // passing arguments by reference
cout << "After swap: x = " << x << ", y = " << y << endl; // values of x and
y are swapped
return 0;
}

```

Output:

Before swap: x = 10, y = 20
 After swap: x = 20, y = 10



Reference variable

- A reference variable is an alias for an existing variable.
- It is declared using the & operator.
- Once a reference is initialized to a variable, it cannot be changed to refer to another variable.
- It must be initialized when declared and cannot be changed to refer to another variable later.
- It does not occupy its own memory space; it shares the memory of the variable it references.

Reference syntax

```

int main() {
    int a = 42;
    int &ref = a; // ref is a reference to a

    cout << "a: " << a << endl; // Output: a: 42
    cout << "ref: " << ref << endl; // Output: ref: 42

    ref = 100; // Changing the value through the reference
    cout << "a after changing ref: " << a << endl; // Output: a after changing
    ref: 100
}

```

```
int b = 200;
// ref = b; // This would not change the reference to refer to b, it would
change the value of a to 200

return 0;
}
```

Inline Functions

- Inline functions are a way to optimize function calls by suggesting to the compiler to replace the function call with the actual code of the function.
- This can reduce the overhead of function calls, especially for small functions that are called frequently.
- The `inline` keyword is used to define an inline function.
- Example of an inline function:

```
#include <iostream>
using namespace std;

inline int square(int x) {
    return x * x;
}

int main() {
    int num = 5;
    // square(num); // function call will be replaced with the actual code of the
function : {return num * num;}
    cout << "Square : " << square(num) << endl;
    return 0;
}
```

Math library functions

- The `<cmath>` header provides a set of functions for performing mathematical operations.

Some commonly used functions include:

- `sqrt(x)`: Returns the square root of `x`.
- `pow(x, y)`: Returns `x` raised to the power of `y`.
- `sin(x)`, `cos(x)`, `tan(x)`: Trigonometric functions (`x` in radians).
- `log(x)`: Returns the natural logarithm (base `e`) of `x`.

etc.

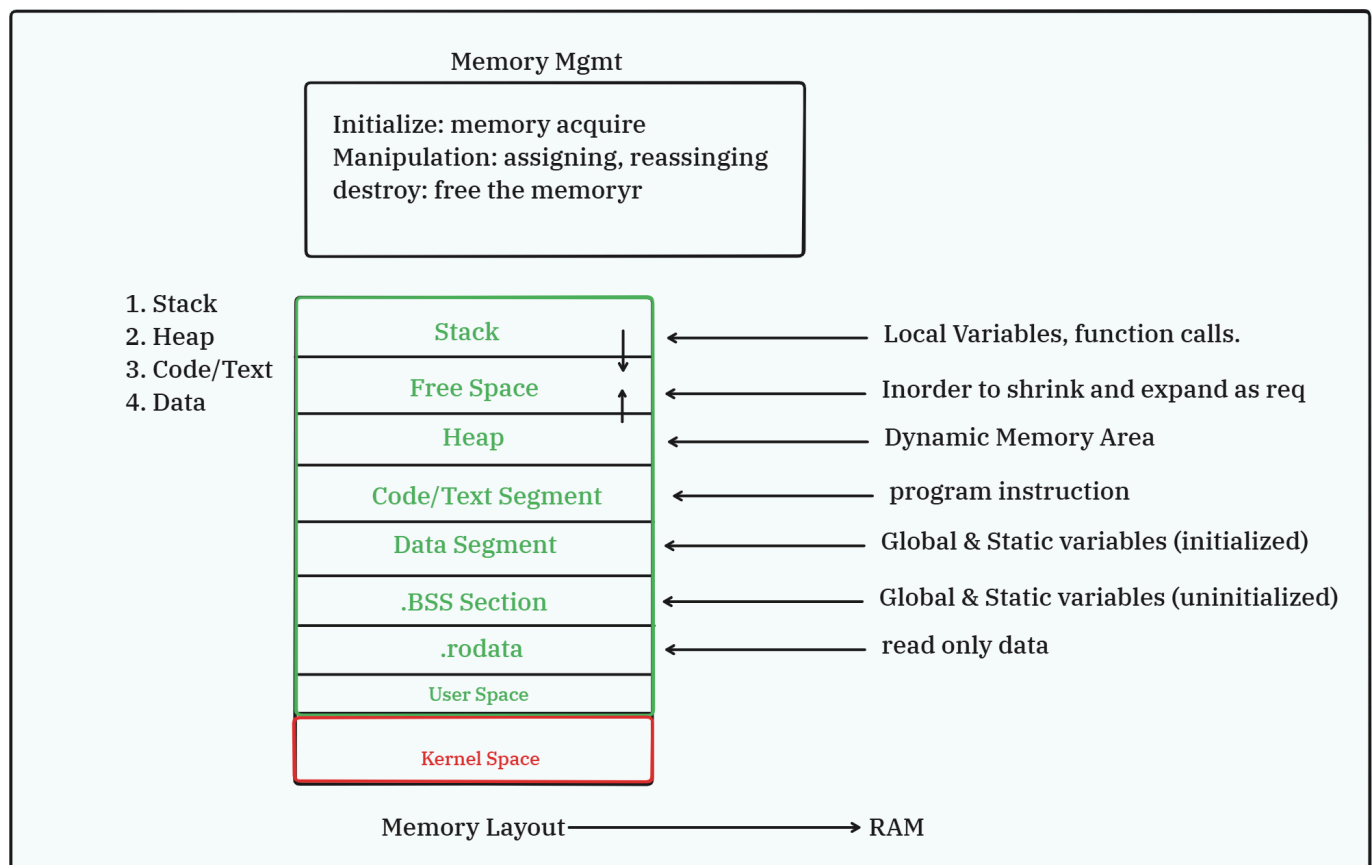
Memory Management in C++

- Memory management in C++ involves the allocation and deallocation of memory during the execution of a program.

- It involves:
 - Initializing variables and objects.
 - Allocating memory
 - Reallocating memory if needed.
 - Reassigning memory to variables or objects.
 - Deallocating memory when it is no longer needed to avoid memory leaks.

Memory layout

- The memory layout of a C++ program typically consists of several segments:
 - **Code Segment:** Contains the compiled code of the program.
 - **Data Segment:** Stores global and static variables.
 - **Heap:** Used for dynamic memory allocation. Memory is allocated and deallocated manually using `new` and `delete`.
 - **Stack:** Used for function call management, local variables, and control flow. Memory is automatically managed (allocated and deallocated) as functions are called and return.



Stack vs Heap conceptually

- **Stack:**
 - Memory is allocated and deallocated automatically.
 - Faster access due to its LIFO (Last In, First Out) nature.
 - Limited in size; can lead to stack overflow if too much memory is used.
 - Used for local variables and function call management.
- **Heap:**
 - Memory is allocated and deallocated manually.
 - Slower access compared to the stack.

- Larger available space.
- Used for dynamic memory allocation.

Scope & lifetime of variables

- **Scope:** The region of the program where a variable is accessible.
- **Lifetime:** The duration for which a variable exists in memory during the execution of a program.

Pointer introduction

- A pointer is a variable that stores the memory address of another variable.
- Pointers are used for dynamic memory allocation, arrays, and for passing large structures or classes to functions efficiently.

Size of pointer variable is 4 bytes in 32-bit system and 8 bytes in 64-bit system.

Address-of operator

- The address-of operator (&) is used to obtain the memory address of a variable.

Example:

```
#include <iostream>
using namespace std;
int main() {
    int a = 10;
    int *ptr = &a; // pointer variable ptr stores the address of a

    cout << a << endl;      //value
    cout << ptr << endl;    //value: it is address of a

    cout << &a << endl;    // address of a
    cout << &ptr << endl;  // address of ptr

    cout << *ptr << endl;   // value of a via ptr

    return 0;
}
```

Pointer declaration

- A pointer is declared by specifying the type of data it points to, followed by an asterisk (*) and the pointer's name.

Example:

```
int *ptr; // ptr is a pointer to an integer
double *dptr; // dptr is a pointer to a double
```

Pointer initialization

- A pointer can be initialized to point to a variable by using the address-of operator (&).

Example:

```
int a = 10;
int *ptr = &a; // ptr is initialized to point to a
```

Dereferencing operator

- The dereferencing operator (*) is used to access the value stored at the memory address held by a pointer.

Example:

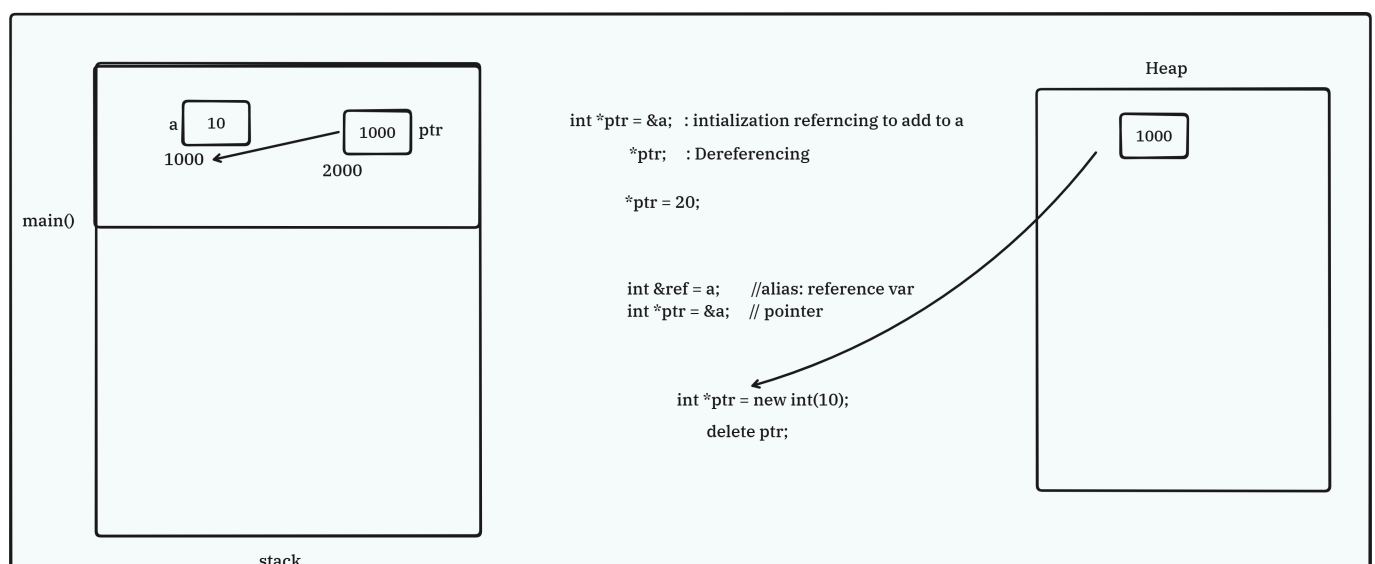
```
int a = 10;
int *ptr = &a;
cout << *ptr << endl; // Output: 10
```

Pointer assignment

- A pointer can be assigned the address of another variable using the address-of operator (&).

Example:

```
int a = 10;
int b = 20;
int *ptr = &a; // ptr points to a
ptr = &b; // ptr now points to b
```



Call by Pointer/address

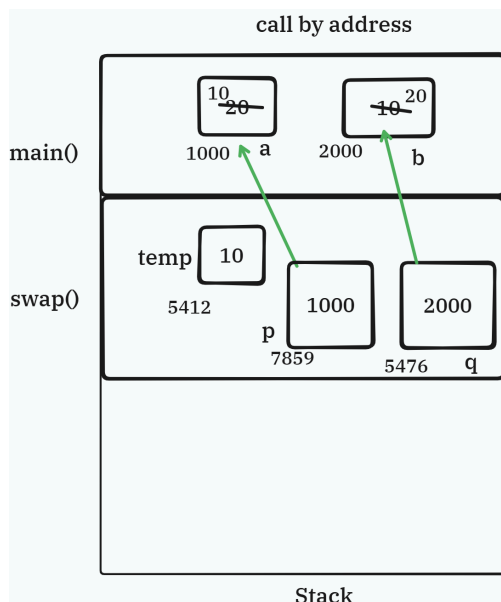
- In call by pointer/address, the address of the actual parameter is passed to the function.

```
#include <iostream>
using namespace std;
void swap(int *a, int *b){
    int temp = *a;
    *a = *b;
    *b = temp;
}

int main() {
    int b = 10, a = 20;
    cout << "Before swap: a = " << a << ", b = " << b << endl;
    swap(&a, &b); // passing addresses of a and b
    cout << "After swap: a = " << a << ", b = " << b << endl; // values of a and
    b are swapped
    return 0;
}
```

Output:

```
Before swap: a = 20, b = 10
After swap: a = 10, b = 20
```



nullptr

- nullptr** is a null pointer constant introduced in C++11.
- It is preferred over **NULL** as it is type-safe and more explicit.

NULL vs nullptr

- `NULL` is a macro that represents a null pointer, typically defined as `0` or `(void*)0`.
- `nullptr` is a keyword that represents a null pointer of any type.

The difference between `NULL` and `nullptr`:

Feature	<code>NULL</code>	<code>nullptr</code>
Type	Can be implicitly converted to integral types (like <code>int</code>)	Cannot be implicitly converted to integral types; it is a pointer type
Usage	Used in older C++ code and C code	Introduced in C++11 for type safety and clarity
Example	<code>int* ptr = NULL;</code>	<code>int* ptr = nullptr;</code>

Tomorrow we will discuss about:

Pointer arithmetic

Pointers and Arrays

Arrays using pointers

Pointer vs Reference

Passing pointers to functions

`const` and Pointer combinations

Pointer to Pointer

Practical pointer problems